

Browser Extension Development Plan

A professional roadmap from first idea to published product

1. Purpose of this plan

A browser extension is a small program built with web technologies (HTML, CSS and JavaScript) that adds features to a browser. This plan describes the ten phases a professional developer or small team follows: validating the idea, scoping the product, designing the architecture, building, securing, testing, publishing and maintaining. It applies to Chrome, Edge, Brave, Opera and Firefox, which all support the Manifest V3 format with only small differences.

2. Plan at a glance

Phase	Goal	Main output
1. Validate	Confirm a real problem and an audience	One page problem statement
2. Scope	Define a small first version	Feature list and permission list
3. Architecture	Choose structure and tools	Component diagram and tech stack
4. Setup	Prepare a clean project	Repository, build tooling, linting
5. Build	Implement the core features	Working extension in developer mode
6. Secure	Protect users and their data	Permission review and privacy policy
7. Test	Find and fix problems	Test reports across browsers
8. Publish	Pass store review	Store listing and signed package
9. Launch	Reach the first users	Landing page and announcements
10. Maintain	Keep it working and improve it	Release cycle and support process

3. Phase 1: Validate the idea

Most failed extensions solve a problem nobody has. Spend a few days on research before writing code.

- Write the problem in one sentence and name the exact person who has it.
- Search the Chrome Web Store, Edge Add ons and Firefox Add ons for similar tools. Read their reviews to learn what users dislike.
- Talk to at least five target users. Ask how they handle the problem today.
- Decide on one feature that makes your extension different from every competitor.

- Define success in numbers, for example 500 active users in three months or a 4.5 star rating.

4. Phase 2: Plan the scope

Aim for a minimum viable product that can be built in two to four weeks. Anything that is not needed for the first release goes on a later list.

Key planning tasks

- Write user stories in the form: as a user, I want to do something so that I get a benefit.
- List the features in three groups: must have, should have and later.
- List the browser permissions you need and justify each one. Request the smallest set possible, because reviewers and users trust extensions that ask for little.
- Decide whether data stays on the device or goes to a server. Local processing is simpler and builds trust.
- Sketch the popup, options page and any on page interface on paper or in a design tool.

5. Phase 3: Design the architecture

A Manifest V3 extension is made of several parts that run in different places and talk to each other through messages.

Component	Where it runs	Typical use
manifest.json	Read by the browser	Declares name, version, permissions, scripts and icons
Service worker	Background, event driven	Handles alarms, network events, context menus and long tasks
Content script	Inside web pages	Reads or changes the page, highlights text, injects interface
Popup	Small window from the toolbar icon	Quick actions and status
Options page	Full browser tab	Settings and account management
Side panel	Panel beside the page	Longer workflows such as notes or assistants
Storage	chrome.storage API	Saves settings and small data, with optional sync across devices

Recommended technology stack

- Language: TypeScript, which catches many errors before the extension runs.
- Build tool: Vite with the CRXJS plugin, or WXT, for fast reload during development.
- Interface: React, Svelte or plain HTML, depending on how complex the screens are.
- Styling: Tailwind CSS or a small custom stylesheet. Use Shadow DOM for injected interface so page styles do not break yours.

- Cross browser support: the webextension polyfill so one codebase works on Chrome and Firefox.
- Machine learning features: TensorFlow.js or ONNX Runtime Web, running models locally inside the extension.

6. Phase 4: Set up the project

- 1 Create a Git repository with a clear folder structure: src, public, tests and docs.
- 2 Initialize the project with npm or pnpm and install the build tool.
- 3 Add ESLint and Prettier so code style stays consistent.
- 4 Configure TypeScript with strict mode turned on.
- 5 Create the manifest with a name, a version number, icons in sizes 16, 32, 48 and 128 pixels, and the first permissions.
- 6 Set up continuous integration that runs linting, tests and a production build on every push.
- 7 Load the unpacked extension in the browser developer mode and confirm that it opens.

7. Phase 5: Build the extension

Work in small vertical slices. Finish one feature completely, including its interface and its error handling, before starting the next.

Core engineering practices

- Keep the service worker stateless. It can be stopped at any time, so save important data in storage instead of variables.
- Use message passing between the popup, content scripts and service worker, and define a typed message format.
- Wrap every browser API call with error handling and sensible fallbacks.
- Make content scripts fast. Avoid scanning the whole page repeatedly and use mutation observers carefully.
- Use a clear naming system and short, well named functions.
- Support dark mode and keyboard navigation from the start.
- Add an onboarding screen that appears on first install and explains what the extension does.
- Prepare for translation by keeping text in locale files from the beginning.

8. Phase 6: Security and privacy

Extensions have powerful access to user data, so stores review them strictly and users judge them quickly.

- Request only the permissions you truly use, and prefer optional permissions that are asked at the moment of need.
- Never run remote code. Manifest V3 requires all code to be inside the package.
- Set a strict content security policy and avoid inline scripts and the use of eval.

- Sanitize any text that you insert into a page to prevent cross site scripting.
- Collect the minimum data. If you collect anything, explain it clearly and offer a way to opt out.
- Write a plain language privacy policy and host it on a public page. Stores require it.
- Keep dependencies updated and run npm audit regularly.
- Never store passwords or secret keys in the extension code.

9. Phase 7: Test and ensure quality

Test type	What it covers	Common tools
Unit tests	Logic functions such as parsers and scoring rules	Vitest or Jest
Integration tests	Messages between popup, content script and service worker	Jest with browser API mocks
End to end tests	Real browser loading the extension and using it	Playwright or Puppeteer
Manual tests	Real websites, slow networks and unusual pages	Test checklist
Performance tests	Memory use, CPU use and page load impact	Browser task manager and DevTools
Accessibility tests	Contrast, labels and keyboard use	Lighthouse and axe

- Test on Chrome, Edge and Firefox, on Windows, macOS and Linux if possible.
- Test on popular and heavy websites, on pages inside iframes and on single page applications.
- Invite five to ten beta users and collect their feedback before the public release.

10. Phase 8: Package and publish

Store submission steps

- 1 Set the final version number and build the production package. Zip the output folder.
- 2 Create a developer account. The Chrome Web Store charges a one time registration fee, while Firefox and Edge are free.
- 3 Prepare store assets: a clear name, a short description, a detailed description, at least three screenshots, a promotional image and an icon.
- 4 Complete the permission justification and data usage forms honestly.
- 5 Submit for review. Reviews usually take from a few hours to a few days.
- 6 If rejected, read the reason carefully, fix the issue and resubmit.

Tips for a strong store listing

- Put the main benefit in the first sentence of the description.
- Use screenshots with short captions that show the extension in real use.

- Include a short demo video if possible.
- Use keywords that people actually search for, but write naturally.

11. Phase 9: Launch and grow

- Create a simple landing page with the problem, a demo and an install button.
- Share the launch on Product Hunt, Reddit communities, developer forums, LinkedIn and relevant social groups.
- Write one or two articles about how and why you built it.
- Ask early users for honest ratings and reviews, and reply to every review.
- Track basic, privacy respecting numbers such as installs, active users and uninstalls.
- Add a feedback link inside the extension so problems reach you directly.

12. Phase 10: Maintain and improve

- Release small updates often, and use semantic version numbers such as 1.2.0.
- Keep a public changelog so users see that the project is alive.
- Watch for browser changes, since browsers update every few weeks and may affect APIs.
- Fix reported bugs within days and tell users when they are fixed.
- Review permissions on every release. Adding a new permission can disable the extension for existing users until they approve it.
- Plan revenue early if needed: a free tier with a paid upgrade, donations or a one time purchase.

13. Suggested timeline for a small team or one developer

Week	Focus	Result
1	Validation, scope and design sketches	Approved plan and feature list
2	Setup and architecture, first working popup	Extension loads and responds
3 to 4	Core features and storage	Usable first version
5	Security review, polish and onboarding	Release candidate
6	Testing, beta feedback and fixes	Stable build
7	Store assets and submission	Listing under review
8	Launch and monitoring	First users and feedback

14. Professional pre launch checklist

Area	Check
Product	One clear purpose and one standout feature

Area	Check
Permissions	Each permission is necessary and explained
Code	No remote code, no eval, no secrets in the package
Privacy	Policy published and data use described accurately
Quality	Tested on major browsers, with no console errors
Performance	Low memory and no visible slowdown on pages
Interface	Dark mode, keyboard support and readable text
Store	Strong name, description, screenshots and icon
Support	Contact email, feedback link and changelog ready

15. Common mistakes to avoid

- Asking for broad permissions such as access to all websites when a narrower option would work.
- Building too many features before getting any user feedback.
- Keeping important state in the service worker memory, which is cleared when the worker stops.
- Ignoring store policies, which leads to rejection or removal.
- Skipping the privacy policy or describing data use inaccurately.
- Abandoning the extension after launch, which causes poor reviews when browsers change.

Useful official resources: the Chrome extensions documentation at developer.chrome.com/docs/extensions, the Mozilla WebExtensions documentation at developer.mozilla.org, and the Microsoft Edge extensions documentation at learn.microsoft.com.